NEXUS

Manual técnico y preparación para el examen H3

Proyecto: POO Nexus / NEXUS v2.0

Autor: Benji Pacheco Lafuente

Lenguaje: C# · **Framework:** .NET 10

Guía de estudio basada en el código fuente del repositorio público.

Incluye arquitectura, POO, ejemplos de código, análisis de ejecución, preguntas razonadas y diagnóstico personal con límites explícitos de evidencia.

Proyecto: POO Nexus / NEXUS v2.0 **Autor:** Benji Pacheco Lafuente **Lenguaje:** C# · **Framework objetivo:** .NET 10 (`net10.0`) **Repositorio:** https://github.com/Benjipacheco69/POO-Nexus **Fuente principal:** `POO Nexus/Program.cs`

> **Criterio de precisión:** las clases principales están declaradas dentro de un único archivo `Program.cs`. Los comentarios que nombran archivos como `Personaje.cs` o `Robot.cs` son separadores lógicos, no archivos independientes. El código revisado utiliza herencia, clases abstractas, métodos virtuales/sobrescritos y `List<Objeto>`. No se encontraron interfaces ni consultas LINQ implementadas en ese archivo; esos temas se presentan como ejemplos complementarios.

Cómo estudiar con este manual sin conexión

1. Descarga este PDF y el repositorio antes de desconectarte.

2. Lee un apartado y localiza en `Program.cs` el método que se menciona.

3. Tapa la respuesta de cada ejercicio y resuélvelo por tu cuenta.

4. Explica el razonamiento en voz alta, como si respondieras al docente.

5. Distingue entre lo que el programa hace actualmente y las mejoras propuestas.

I. Arquitectura y programación orientada a objetos

1. Propósito y funcionamiento general

NEXUS es un videojuego de exploración y supervivencia por consola ambientado en el año 2297. El jugador controla a un cadete que explora una cuadrícula, administra energía y estabilidad, recoge objetos y pistas relacionadas con IRIS, se enfrenta a enemigos y robots y debe cumplir objetivos antes de regresar a extracción.

El punto de entrada es `Program.Main()`: crea un objeto `Juego` y llama a `Iniciar()`. La clase `Juego` coordina la partida: crea al cadete, utiliza el mapa y la misión, presenta el menú, procesa las opciones y resuelve eventos y encuentros. La partida puede terminar por victoria, salida voluntaria o pérdida de recursos, según las condiciones que evalúa el código.

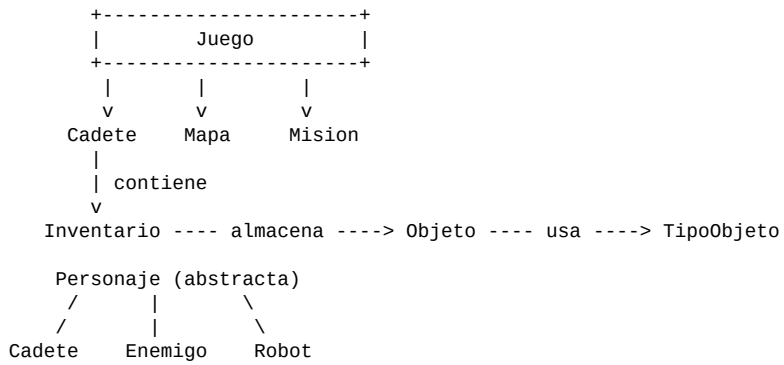
2. Estructura y responsabilidades de las clases

Tipo	Responsabilidad
`Program`	Punto de entrada; instancia `Juego` e inicia la aplicación.
`Juego`	Coordina menú, exploración, eventos, combates, mapa y misión.
`Personaje`	Clase abstracta base con nombre, energía, ataque y comportamiento común.
`Cadete`	Jugador; administra estabilidad, XP, escudo, inventario, pistas y exploración.
`Enemigo`	Entidad que contraataca y devuelve 20 XP como recompensa.
`Robot`	Entidad con defensa que reduce a la mitad el daño recibido; recompensa de 30 XP.
`Objeto`	Modela nombre, tipo y valor de un objeto.
`TipoObjeto`	Enumeración: `Celula`, `Escudo`, `ModuloAtaque` y `Analizador`.
`Inventario`	Almacena `List<Objeto>`, limita la capacidad y agrega, consulta, busca y elimina objetos.
`Mapa`	Controla cuadrícula, posición, movimiento y ubicaciones relevantes.

Tipo	Responsabilidad
`Mision`	Registra objetivos, progreso y recompensa de la misión.

Ubicación: todas estas clases aparecen en `POO Nexus/Program.cs`; no asumir que existe un archivo separado por cada nombre de clase.

3. Diagrama de clases



`Cadete`, `Enemigo` y `Robot` heredan de `Personaje`. `Cadete` mantiene un `Inventario`, y `Inventario` mantiene una colección de objetos. `Juego` coordina los componentes; esto es una relación de uso/colaboración y no significa que todos hereden entre sí.

4. Programación estructurada frente a POO

La programación estructurada organiza el programa con secuencias, condiciones (`if`, `switch`), ciclos (`while`, `for`) y procedimientos. La programación orientada a objetos organiza el estado y las operaciones en clases y objetos que colaboran.

NEXUS combina ambas: su diseño utiliza clases y objetos, mientras que los métodos emplean condiciones y ciclos para controlar la ejecución. POO no elimina las estructuras de control; permite agrupar los datos y las reglas que actúan sobre ellos.

II. Fundamentos de clases y objetos

5. Clases, objetos e instanciación

Una **clase** define una estructura y un comportamiento. Un **objeto** es una instancia concreta de esa clase. La palabra `new` solicita crear una instancia.

```
Juego juego = new Juego();
juego.Iniciar();
```

En este ejemplo, `Juego` es el tipo, `juego` es la variable que guarda una referencia y `new Juego()` crea el objeto. La llamada `Iniciar()` ejecuta un método de ese objeto.

6. Campos, propiedades, atributos y métodos

En el uso académico habitual, “atributo” puede referirse a un dato de un objeto; en C#, el término preciso para una variable declarada dentro de una clase es **campo**. Una **propiedad** ofrece una forma controlada de leer o asignar un valor. Un **método** define una operación.

En `Personaje` existen campos privados como `nombre`, `energia` y `ataque`, y métodos como `GetEnergia()`, `PerderEnergia()` y `RecuperarEnergia()`. El código actual usa principalmente campos y métodos de acceso, no propiedades C# para esos valores.

Ejemplo complementario de propiedad:

```
public int Energia { get; private set; }
```

`get` permite consultar la propiedad y `private set` limita la asignación directa desde fuera de la clase.

7. Constructores, sobrecarga, `this` y `base`

Un constructor inicializa un objeto cuando se crea. En `Personaje`, el constructor recibe nombre, energía y ataque. El constructor de `Cadete` usa `base(nombre, energia, 10)` para inicializar la parte heredada y `this.estabilidad = estabilidad` para diferenciar el campo del parámetro.

- `this` se refiere al objeto actual.
- `base(...)` llama al constructor de la clase base.
- `base.RecibirDano(dano)` llama a la implementación heredada de ese método.

La **sobrecarga** consiste en definir varios métodos o constructores con el mismo nombre y diferentes listas de parámetros. No se observa una sobrecarga de constructores en las clases principales revisadas; el siguiente es un ejemplo complementario:

```
class Registro
{
    public string Nombre { get; }

    public Registro(string nombre) => Nombre = nombre;
    public Registro(string nombre, int codigo) => Nombre = nombre + " #" + codigo;
}
```

8. Encapsulamiento y modificadores de acceso

El encapsulamiento protege el estado interno y hace que los cambios pasen por operaciones que pueden aplicar reglas. En `Personaje`, `PerderEnergia()` impide que la energía quede por debajo de cero y `RecuperarEnergia()` impide que supere 100.

- `private`: accesible dentro del tipo que lo declara.
- `public`: accesible desde código que puede ver el tipo.
- `protected`: accesible desde la clase y sus clases derivadas.
- `internal`: accesible dentro del mismo ensamblado.

El encapsulamiento no consiste solamente en poner campos como `private`; también consiste en definir operaciones coherentes y proteger las invariantes del objeto.

III. Herencia, polimorfismo e interfaces

9. Herencia y especialización

`Cadete`, `Enemigo` y `Robot` derivan de `Personaje`. Así reutilizan nombre, energía, ataque y operaciones comunes. Cada clase añade o especializa comportamientos: el cadete tiene inventario y escudo; el enemigo contraataca; el robot activa una defensa.

La herencia representa una relación “es un”: un `Robot` es un `Personaje`. No debe usarse únicamente para compartir código si la relación conceptual no tiene sentido.

10. `virtual`, `override` y polimorfismo

`Personaje` declara métodos virtuales como `RecibirDano()`, `AlAparecer()` y `Reaccionar()`. Las clases derivadas pueden redefinirlos con `override`.

```
Personaje entidad = new Robot("R-1", 30, 6);
entidad.AlAparecer();
entidad.RecibirDano(5);
```

Aunque la variable se declara como `Personaje`, el objeto real es un `Robot`; por ello se ejecuta `Robot.RecibirDano()`. Esa selección de comportamiento en tiempo de ejecución es polimorfismo.

El robot activa su defensa y calcula `dano = dano / 2`. Como ambos valores son enteros, `5 / 2` produce 2 en C#; la fracción se trunca. La energía pasa de 30 a 28. En cambio, `Cadete.RecibirDano()` intenta absorber primero el daño con su escudo y llama a `base.RecibirDano()` si queda daño.

11. Clases y métodos abstractos

`Personaje` es abstracta, por lo que no se instancia directamente. Declara `MostrarInformacion()` como abstracto, sin cuerpo. Las clases concretas derivadas implementan ese método para mostrar los datos apropiados a cada tipo.

Una clase abstracta puede incluir campos, constructores y métodos implementados además de métodos abstractos. Su propósito es definir una base común incompleta que las clases derivadas especializan.

12. Interfaces y comparación con composición

No se encontró una interfaz implementada en el código revisado de NEXUS. Este ejemplo es complementario:

```
interface IAnalizable
{
    string ObtenerInforme();
}

class Sensor : IAnalizable
{
    public string ObtenerInforme() => "Lectura del sensor";
}
```

La interfaz define un contrato: cualquier clase que la implemente debe cumplir ese miembro. Una clase puede implementar varias interfaces.

Mecanismo	Qué expresa	Ejemplo
Herencia	“Es un” tipo más específico; reutiliza y especializa comportamiento.	`Robot : Personaje`
Composición	“Tiene un” componente que colabora con el objeto.	`Cadete` contiene `Inventario`.
Interfaz	“Puede hacer” una operación definida por un contrato.	Ejemplo complementario `IAnalizable`.

No se debe afirmar que NEXUS ya utiliza interfaces: el ejemplo anterior sirve para estudiar el tema y podría incorporarse en una futura mejora.

IV. Colecciones y LINQ

13. `List<T>` en NEXUS

`Inventario` declara `List<Objeto> objetos` y la inicializa con `new List<Objeto>()`. La lista crece dinámicamente; la capacidad máxima de ocho objetos la impone la lógica del programa, no el tipo `List<T>` por sí solo.

`Agregar()` verifica si `objetos.Count >= capacidad` antes de llamar a `Add()`. `Consultar()` recorre la colección con `for`. Los métodos que reciben una posición visible al jugador deben convertirla al índice de C#: si la opción elegida es 1, el índice de la lista es 0.

```
for (int i = 0; i < objetos.Count; i++)
{
    Console.WriteLine(objetos[i].GetDescripcion());
}
```

Este recorrido visita cada elemento desde el índice cero hasta `Count - 1`.

14. LINQ y expresiones lambda (ejemplo complementario)

El archivo `Program.cs` revisado no contiene consultas LINQ ni expresiones lambda. El siguiente ejemplo es complementario, no una funcionalidad existente:

```
var descripciones = objetos
    .Where(o => o.GetValor() >= 20)
    .OrderBy(o => o.GetValor())
    .Select(o => o.GetDescripcion());

foreach (string descripcion in descripciones)
{
    Console.WriteLine(descripcion);
}
```

- `Where` filtra los elementos que cumplen una condición.
- `OrderBy` ordena según una clave.
- `Select` transforma cada elemento en otro valor.
- `o => ...` es una expresión lambda: describe una función breve que recibe `o`.
- `var` permite que el compilador infiera el tipo de la variable.

Para usar LINQ con colecciones, normalmente se importa `System.Linq`. Un recorrido tradicional con `foreach` e `if` también puede filtrar; LINQ expresa la intención de forma declarativa. Muchas consultas LINQ se ejecutan al enumerar sus resultados, por ejemplo con `foreach` o `ToList()`.

V. Análisis del funcionamiento

15. Flujograma del ciclo general

```
[Inicio: Main]
      |
      v
[Crear Juego e iniciar]
      |
      v
[Solicitar datos del cadete]
      |
      v
{¿Cumple los requisitos?} -- No --> [Rechazar inicio / salir]
      |
     Sí
      v
[Crear mapa, misión y objetos iniciales]
```

```
      |
      v
[Mostrar menú y leer opción]
      |
      v
[Procesar acción / explorar / combatir]
      |
      v
{¿La partida terminó o se agotaron recursos?}
      | No                          | Sí
      +------> [Volver al menú]     v
                            [Mostrar resultado]
```

Este flujograma resume el flujo de alto nivel. Para validar cada detalle, localiza `Main()`, `Juego.Iniciar()` y el método que procesa las opciones en `Program.cs`.

16. Seguimiento del estado de un objeto

Supongamos que el cadete empieza con 60 puntos de energía y utiliza una célula cuyo valor es 25. En `Cadete.UsarObjeto()`, se obtiene el objeto, se llama a `RecuperarEnergia(25)` y, si el uso es válido, se elimina del inventario.

Paso	Operación	Energía	Célula
1	Estado inicial supuesto	60	Disponible
2	Recuperar 25 puntos	85	Disponible durante la operación
3	Consumir objeto	85	Eliminada del inventario

Si la energía inicial fuera 90, el resultado sería 100, no 115, porque `RecuperarEnergia()` limita el máximo. Los valores iniciales de este ejercicio son supuestos para demostrar la regla, no el resultado de una partida concreta.

17. Predicción del resultado de código

```
Personaje entidad = new Robot("R-1", 30, 6);
entidad.AlAparecer();
entidad.RecibirDano(5);
Console.WriteLine(entidad.GetEnergia());
```

Resultado razonado: `AlAparecer()` activa la defensa del robot. El daño 5 se divide entre 2 usando enteros y queda en 2. La energía pasa de 30 a 28. La última línea imprime 28, además de los mensajes que los métodos escriben en consola.

18. ¿Qué ocurre si se modifica una condición?

En `Inventario.Agregar()`, la condición correcta para rechazar un nuevo objeto cuando ya se alcanzó la capacidad es:

```
if (objetos.Count >= capacidad)
{
    Console.WriteLine("Inventario lleno.");
    return false;
}
```

Si se cambia >= por >, cuando `Count` sea exactamente igual a `capacidad`, la condición será falsa y se agregará un objeto adicional. El inventario podrá superar el límite por una unidad. Este cambio rompe una regla del objeto, llamada invariante: la cantidad no debe superar la capacidad.

19. Observaciones técnicas y mejoras posibles

• Los campos de `Personaje` son privados y se modifican mediante métodos que aplican límites.

• `Robot.RecibirDano()` usa división entera; el daño impar pierde la fracción.

• El proyecto concentra las clases en un único `Program.cs`. Separarlas en archivos facilitaría la lectura y el mantenimiento sin cambiar necesariamente el comportamiento.

• `LeerEntero()` y las validaciones de entrada deben revisarse para asegurar que todos los valores aceptados tengan sentido en el juego.

• Debe comprobarse si escapar de un encuentro con el guardián debería contar como objetivo cumplido. Si el requisito es derrotarlo, la misión no debería marcarlo cumplido al escapar.

• No se observaron interfaces ni LINQ en el código revisado; deben poder explicarse con ejemplos independientes.

• Antes de modificar el proyecto original, crea una rama o copia de trabajo y prueba el cambio.

VI. Preparación para el examen H3

Pregunta 1 — Comprensión de POO

Pregunta: ¿Por qué `Juego` puede mostrar el estado del cadete sin leer directamente todos sus campos privados?

Respuesta razonada: porque puede llamar al método público `MostrarInformacion()` de `Cadete`. Ese método pertenece al objeto y puede acceder a sus propios campos privados. Así `Juego` no depende de cómo se almacenan internamente los datos y se mantiene el encapsulamiento.

Pregunta 2 — Clases, objetos y constructores

Pregunta: ¿Qué hace `new Inventario(8)` y qué ocurriría si se creara `new Inventario(2)`?

Respuesta razonada: crea un objeto `Inventario` y pasa 8 al constructor, que guarda ese valor como capacidad e inicializa la lista vacía. Si se pasa 2, la lógica de `Agregar()` debería permitir como máximo dos objetos antes de rechazar otro. La capacidad se controla mediante el campo `capacidad`, no por una restricción automática de `List<T>`.

Pregunta 3 — Herencia y polimorfismo

Pregunta: Si una variable de tipo `Personaje` apunta a un objeto `Robot`, ¿qué método se ejecuta al llamar `RecibirDano(5)` y por qué?

Respuesta razonada: se ejecuta `Robot.RecibirDano(5)`, porque el método es virtual en la base y está sobrescrito con `override`. El tipo real del objeto determina la implementación. El robot activa su regla defensiva, reduce el daño de 5 a 2 mediante división entera y luego llama a la implementación base para restar la energía.

Pregunta 4 — Interfaces y diseño

Pregunta: ¿Por qué una interfaz `IAnalizable` podría ser útil si más adelante hubiera sensores, robots y registros que debieran producir informes?

Respuesta razonada: permitiría que tipos distintos ofrecieran la misma operación contractual sin tener que heredar todos de una única clase base. El código consumidor podría depender de `IAnalizable` y llamar a `ObtenerInforme()` sin conocer la clase concreta. Es una propuesta complementaria; no se afirma que ya exista en NEXUS.

Pregunta 5 — Listas genéricas y LINQ

Pregunta: ¿Qué diferencia hay entre recorrer `List<Objeto>` con `for` y filtrar sus elementos con `Where()`?

Respuesta razonada: con `for` se controla explícitamente el índice y la condición de recorrido. `Where()` expresa declarativamente el predicado que deben cumplir los elementos. Ambos pueden filtrar datos, pero LINQ puede hacer la intención más legible. En el código actual se usa un recorrido tradicional; la consulta LINQ de este manual es complementaria.

Pregunta 6 — Predicción y modificación de código

Pregunta: Un robot tiene 30 de energía, activa la defensa y recibe 7 puntos de daño. ¿Cuánta energía queda? ¿Qué cambiaría si se usara división decimal?

Respuesta razonada: con el código actual, `7 / 2` es división entera y produce 3; quedan 27 puntos de energía. Si se cambia el cálculo a decimal, el diseño tendría que definir cómo representar energía y daño fraccionarios o cómo redondearlos. No basta con cambiar el literal: hay que revisar los tipos y las reglas del juego.

Pregunta 7 — Encapsulamiento e invariantes

Pregunta: ¿Por qué `RecuperarEnergia()` limita la energía a 100 en lugar de permitir cualquier suma?

Respuesta razonada: porque mantiene una regla válida del dominio: la energía no supera 100. Si cada clase modificara directamente el campo, podría saltarse la regla y crear estados imposibles. Centralizar la operación protege la consistencia del objeto.

Pregunta 8 — Depuración y análisis de condiciones

Pregunta: Si `Inventario.Agregar()` compara `objetos.Count > capacidad`, ¿qué error aparece y cómo se corrige?

Respuesta razonada: cuando el inventario está exactamente lleno, `Count == capacidad`, la condición > es falsa y se permite agregar un elemento extra. Se corrige usando >=, que rechaza la inserción cuando la cantidad ya alcanzó la capacidad.

Pregunta 9 — Composición frente a herencia

Pregunta: ¿Por qué `Cadete` contiene un `Inventario` en vez de heredar de él?

Respuesta razonada: un cadete tiene un inventario; no es un inventario. La composición modela esa relación y permite que `Cadete` delegue operaciones de almacenamiento al componente. La herencia se reserva para una relación de especialización como `Robot` es un `Personaje`.

Pregunta 10 — Análisis crítico del proyecto

Pregunta: Menciona una mejora que harías sin cambiar el objetivo principal del juego y explica cómo comprobarías que no lo rompiste.

Respuesta razonada: separaría las clases en archivos individuales para mejorar el mantenimiento. Después compilaría el proyecto, ejecutaría los principales flujos (inicio válido e inválido, explorar, recoger/usar objetos, combate y finalización) y compararía los resultados. Una refactorización debe preservar el comportamiento esperado.

Repaso rápido: conceptos que debes poder explicar

• **Clase:** definición de datos y operaciones.

• **Objeto:** instancia concreta de una clase.

• **Constructor:** inicializa una instancia.

• **Campo:** variable declarada dentro de un tipo.

• **Propiedad:** acceso controlado a un valor mediante `get/set`.

• **Encapsulamiento:** protege el estado y centraliza reglas.

• **Herencia:** una clase deriva de otra y especializa comportamiento.

• **Polimorfismo:** una llamada a un método virtual se resuelve según el tipo real del objeto.

• **Abstract:** define una base incompleta o un miembro que debe implementar una clase derivada.

• **Interfaz:** contrato de miembros que una clase se compromete a implementar.

• **Composición:** un objeto contiene o utiliza otro objeto.

• `List<T>`**:** colección genérica dinámica.

• **Lambda:** función breve, por ejemplo `o => o.GetValor() >= 20`.

• **LINQ:** operaciones declarativas como `Where`, `Select` y `OrderBy`.

Diagnóstico personal de aprendizaje — Prompt espejo

Alcance y límites de la evaluación

Este diagnóstico se basa únicamente en las evidencias disponibles en la conversación utilizada para preparar este documento y en el proceso de consulta del repositorio. No equivale a una evaluación presencial ni demuestra por sí solo dominio independiente del código. Las explicaciones generadas por el asistente no se atribuyen al estudiante como si fueran respuestas propias. Para una evaluación más sólida, el estudiante debe responder los ejercicios sin ayuda y guardar sus respuestas.

1. Fortalezas demostradas

• Se identifica un objetivo práctico concreto: publicar el proyecto de Programación II, documentarlo y prepararlo para el Hito 3.

• Se ha avanzado en el uso de GitHub y GitHub Desktop, y se mantiene un repositorio público con código, README y documentación.

• El repositorio tiene una estructura inicial de documentación (`docs/`) y un proyecto C# que puede consultarse.

• En la conversación disponible no hay suficientes explicaciones conceptuales escritas por el estudiante para afirmar que ya domina de manera independiente la herencia, el polimorfismo, las interfaces o LINQ. Esta ausencia de evidencia no demuestra incapacidad; indica que hace falta observar su razonamiento propio.

2. Falencias observadas

• No hay respuestas propias suficientes a ejercicios de predicción o explicación del código que permitan identificar errores conceptuales concretos.

• No se ha demostrado todavía, mediante una explicación independiente, cómo se resuelve una llamada virtual, cómo se protege una invariante o cómo se calcula el resultado de una consulta LINQ.

• El uso de herramientas de documentación debe acompañarse de verificación directa del código; diagramas y explicaciones automáticas pueden omitir detalles o equivocarse.

• No se puede concluir que existan dificultades recurrentes en conceptos específicos porque la conversación no contiene una serie suficiente de intentos, errores y correcciones del estudiante.

3. Diagnóstico del Hito 3

Concepto	Clasificación	Justificación
POO	Evidencia insuficiente	Se ha solicitado preparar documentación de POO, pero no hay una explicación conceptual propia suficiente para verificar dominio independiente.
Clases y objetos	Evidencia insuficiente	El repositorio contiene clases y objetos; falta una predicción o explicación propia del estudiante.
Constructores	Evidencia insuficiente	No hay una respuesta independiente que explique parámetros, inicialización y llamada a `base`.
Encapsulamiento	Evidencia insuficiente	El código contiene campos privados y métodos de control, pero el estudiante no ha explicado todavía por qué protegen el estado.
Herencia	Evidencia insuficiente	La estructura de herencia está presente en el código; falta una explicación propia de la relación base-derivada.
Polimorfismo	Evidencia insuficiente	No hay una predicción independiente de la llamada a `RecibirDano()` según el tipo real del objeto.
Interfaces	Evidencia insuficiente	No se identificó una interfaz implementada en el código y aún no hay respuesta propia al ejemplo complementario.
Listas genéricas	Evidencia insuficiente	El proyecto utiliza `List<Objeto>`, pero falta demostrar sin ayuda cómo operan índices, `Count`, capacidad y eliminación.
LINQ	Evidencia insuficiente	No se encontraron consultas LINQ en el código revisado y no existe una resolución propia de `Where`, `OrderBy` y `Select`.

Interpretación: “evidencia insuficiente” no significa que el estudiante desconozca el tema; significa que la conversación no aporta pruebas suficientes para clasificarlo como comprensión demostrada, parcial o dificultad observable.

4. Forma de aprender

La evidencia muestra un enfoque orientado a completar un producto real: consultar el repositorio, organizar archivos y pedir ayuda para preparar la documentación. Esto permite trabajar sobre un contexto concreto en lugar de estudiar únicamente definiciones abstractas.

Para demostrar aprendizaje autónomo, el siguiente paso es contrastar cada explicación con el código, ejecutar ejemplos, registrar predicciones antes de ejecutarlos y corregir razonamientos cuando los resultados no coincidan. No hay evidencia suficiente para evaluar todavía la frecuencia con que el estudiante verifica resultados o resuelve errores sin ayuda.

5. Plan de estudio prioritario

Prioridad 1 — Polimorfismo. Sin ejecutar el código, predice qué ocurre cuando una referencia `Personaje` apunta a un `Robot` y recibe daño 5. Escribe el orden de llamadas y calcula la energía final.

Prioridad 2 — Encapsulamiento e invariantes. Explica por qué `RecuperarEnergia()` no permite superar 100. Modifica una copia para retirar el límite y describe qué estado inválido podría producirse.

Prioridad 3 — Constructores, `this` y `base`. Dibuja el orden de inicialización de `Cadete` y `Personaje`. Explica por qué el constructor derivado llama a `base(nombre, energia, 10)`.

Prioridad 4 — `List<T>` y LINQ. Crea cinco objetos con valores diferentes; filtra los de valor mayor o igual a 20, ordénalos y selecciona sus descripciones. Resuelve el mismo problema primero con `foreach` e `if`.

Prioridad 5 — Interfaces y diseño. Diseña una interfaz `IAnalizable`, impleméntala en dos clases distintas y explica cómo un método puede recibir cualquier objeto que cumpla ese contrato.

Para cada ejercicio, escribe primero una solución propia, ejecútala después y registra qué corregiste. Esa evidencia permitirá reemplazar la clasificación de “evidencia insuficiente” por una evaluación mejor sustentada.

6. Conclusión

Se ha demostrado progreso en la organización y documentación del proyecto y en el uso de un repositorio público. El código ofrece ejemplos reales para estudiar abstracción, encapsulamiento, herencia, polimorfismo, composición y listas genéricas. Sin embargo, la conversación disponible no basta para afirmar que todos esos conceptos están dominados de forma independiente.

Antes del examen, el estudiante debería poder explicar cada clase sin leer el manual, predecir la salida de un fragmento, justificar el resultado con las reglas de C#, detectar una condición incorrecta y escribir un ejemplo sencillo de interfaz y LINQ sin asistencia. El diagnóstico debe actualizarse después de observar esas respuestas propias; no sería correcto inventar evidencias que todavía no existen.

Lista final antes de entregar en Moodle

• [ ] Verificar que el repositorio sea público y que los archivos fuente estén accesibles.

• [ ] Abrir el proyecto con el SDK de .NET 10, compilarlo y probar el inicio.

• [ ] Revisar que `.gitignore` excluya `.vs/`, `bin/`, `obj/` y otros archivos generados, y comprobar que no haya secretos.

• [ ] Guardar el manual Markdown y el PDF dentro de `docs/`.

• [ ] Revisar el PDF sin conexión y confirmar que tablas, ejemplos y diagramas sean legibles.

• [ ] Estudiar las preguntas sin mirar las respuestas y anotar las dudas reales.

• [ ] Entregar en Moodle el enlace público del repositorio y el PDF.

• [ ] Si el docente exige un diagnóstico basado en la conversación completa tras utilizar el prompt exacto, ejecutar ese prompt en el chat de trabajo y sustituir este diagnóstico preliminar por el resultado real, sin atribuir al estudiante las respuestas del asistente.

Referencia técnica

• Repositorio del proyecto: https://github.com/Benjipacheco69/POO-Nexus

• Guía oficial de GitHub Desktop para añadir y publicar un proyecto: https://docs.github.com/en/desktop/adding-and-cloning-repositories/adding-an-existing-project-to-github-using-github-desktop